

Problem 1 – Smart Temperature Converter 🌡️

Difficulty

★ Beginner

Overview

A local travel company receives tourists from different countries. Some visitors are familiar with **Celsius**, while others use **Fahrenheit**. The company wants a simple application that quickly converts temperatures between the two units and displays the result in a friendly way.

Your task is to build a smart temperature converter that can be used repeatedly for different customers.

Mission (Python)

1. Store the available temperature units.

The application should support the following units:

Unit

Celsius

Fahrenheit

2. Make the application run forever.

For every customer, ask for:

- The temperature value.
 - The current temperature unit.
 - The unit they want to convert to.
-

3. Validate the user's input.

The application should ensure that:

- The entered unit exists.
- The source and destination units are not the same.

If an invalid option is entered, ask the user to try again.

4. Convert the temperature.

Use the correct mathematical formula depending on the selected conversion.

Supported conversions:

- Celsius → Fahrenheit
 - Fahrenheit → Celsius
-

5. Display the conversion result.

Example:

```
=====
Conversion Result
=====
Temperature : 25°C

Converted To : Fahrenheit

Result : 77.0°F
=====
```

6. Ask whether another customer wants to use the application.

If the answer is:

- **Yes** → start another conversion.
 - **No** → display a goodbye message and end the program.
-

Bonus Mission ★

Add one more temperature unit:

- Kelvin

Your program should support converting between:

- Celsius ↔ Fahrenheit
 - Celsius ↔ Kelvin
 - Fahrenheit ↔ Kelvin
-

Useful Hints

- Store the units using a **list** or **dictionary**.
- Use **functions** to separate each conversion.
- Use **while True** to keep the program running.
- Use **if / elif / else** to determine which formula to use.
- Use **.lower()** to make input validation easier.
- Round the result to **2 decimal places**.

Expected Console Output

```
=====
🌡 Smart Temperature Converter
=====
```

Supported Units

- Celsius
- Fahrenheit

Enter temperature: 30

Current unit: Celsius

Convert to: Fahrenheit

```
=====
Conversion Result
=====
30°C = 86.0°F
```

Would you like another conversion? (yes/no): yes

GUI Mission (Streamlit)

Create a graphical version of the application.

Required Text Functions

Use:

- `st.title()`
 - `st.header()`
 - `st.write()`
 - `st.divider()`
-

Required Input Widgets

Your application **must** use:

- `st.number_input()` → Temperature value
 - `st.selectbox()` → Current unit
 - `st.selectbox()` → Convert to
 - `st.button()` → Convert
-

Required Status Elements

Use at least **three** of the following:

- `st.info()` → Display supported units
 - `st.success()` → Successful conversion
 - `st.warning()` → Same unit selected
 - `st.error()` → Invalid conversion (if applicable)
-

Required Layout

Your application **must** include:

- **Sidebar**
 - Instructions
 - Supported units
 - **Two Columns**
 - Left → User input
 - Right → Conversion result
-

Expected GUI

 Smart Temperature Converter

Sidebar

Instructions

Supported Units

- Celsius
 - Fahrenheit
-

Temperature

[30]

Current Unit

[Celsius ▼]

Convert To

[Fahrenheit ▼]

[Convert]

Conversion Result

30°C

↓

86°F

 Conversion completed successfully!

Python Topics Covered

- Variables
 - Type casting
 - Integers & floats
 - Arithmetic operators
 - Comparison operators
 - Conditional statements (if, elif, else)
 - while loop
 - User-defined functions
 - Lists or dictionaries
 - String methods (.lower())
 - Input validation
-

Streamlit Topics Covered

Displaying Text

- `st.title()`
- `st.header()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.number_input()`
- `st.selectbox()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
 - `st.columns()`
-

Learning Objectives

After completing this project, you should be able to:

- Design a simple console application.
- Break a problem into smaller functions.
- Validate user input.
- Use loops to repeat a program.
- Build your first interactive Streamlit application.
- Organize a basic Streamlit interface using a sidebar and columns.

This problem is intentionally simple and focuses on mastering the core concepts before moving on to more complex applications like calculators, quiz games, and management systems.